

**LAPORAN P10**

**IMPLEMENTASI SECURE SHELL (SSH) PADA SERVER**  
**PROXMOX VE MENGGUNAKAN PuTTY**



Dosen Pengajar:  
Muhamad Ainurohman, S. Kom

Disusun Oleh:

Nama : Dea Ajeng Aulia  
NIM : 2402015  
Kelas : Teknik Informatika 4A

**PROGRAM STUDI D3 TEKNIK INFORMATIKA**  
**POLITEKNIK PURBAYA**  
**2026/2027**

## KATA PENGANTAR

Segala puji dan syukur penulis ucapkan kepada Tuhan Yang Maha Esa karena atas berkat, rahmat, dan penyertaan-Nya, kegiatan praktikum serta penyusunan laporan yang berjudul **"Implementasi Secure Shell (SSH) pada Server Proxmox VE Menggunakan PuTTY"** dapat diselesaikan dengan lancar.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum pada mata kuliah **Penjaminan dan Keamanan Informasi**. Selain itu, laporan ini bertujuan untuk mendokumentasikan proses implementasi serta memberikan pemahaman mengenai penggunaan protokol Secure Shell (SSH) sebagai media akses jarak jauh yang aman pada server Proxmox VE dengan bantuan aplikasi PuTTY.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penyusunan maupun isi. Oleh sebab itu, penulis mengharapkan kritik dan saran yang bersifat membangun agar laporan ini dapat menjadi lebih baik di masa mendatang.

Tegal, 30 Juni 2026  
Penulis,

Dea Ajeng Aulia

## DAFTAR ISI

|                                                    |           |
|----------------------------------------------------|-----------|
| KATA PENGANTAR .....                               | ii        |
| DAFTAR ISI.....                                    | iii       |
| <b>BAB I.....</b>                                  | <b>1</b>  |
| <b>PENDAHULUAN.....</b>                            | <b>1</b>  |
| 1.1 Latar Belakang .....                           | 1         |
| 1.2 Tujuan .....                                   | 1         |
| 1.3 Manfaat .....                                  | 2         |
| <b>BAB II .....</b>                                | <b>3</b>  |
| <b>DASAR TEORI.....</b>                            | <b>3</b>  |
| 2.1 Virtualisasi .....                             | 3         |
| 2.2 Proxmox Virtual Environment (Proxmox VE) ..... | 4         |
| 2.3 Secure Shell (SSH).....                        | 5         |
| 2.4 PuTTY .....                                    | 6         |
| 2.5 Oracle VirtualBox .....                        | 7         |
| 2.6 Virtual Machine (VM) .....                     | 9         |
| 2.7 Linux Container (LXC).....                     | 9         |
| 2.8 Konfigurasi Jaringan pada Proxmox VE .....     | 10        |
| <b>BAB III.....</b>                                | <b>12</b> |
| <b>PRAKTIKUM .....</b>                             | <b>12</b> |
| 3.1 Alat dan Bahan.....                            | 12        |
| 3.2 Prosedur Praktikum.....                        | 12        |
| <b>BAB IV .....</b>                                | <b>22</b> |
| <b>HASIL DAN PEMBAHASAN .....</b>                  | <b>22</b> |
| 4.1 Hasil Praktikum.....                           | 22        |

|                      |           |
|----------------------|-----------|
| 4.2 Pembahasan.....  | 23        |
| <b>BAB V.....</b>    | <b>26</b> |
| <b>PENUTUP.....</b>  | <b>26</b> |
| 5.1 Kesimpulan ..... | 26        |
| 5.2 Saran.....       | 27        |
| DAFTAR PUSTAKA ..... | 28        |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Kemajuan teknologi di bidang komputer dan jaringan telah mengubah cara pengelolaan infrastruktur teknologi informasi menjadi lebih fleksibel. Saat ini, administrator sistem tidak harus berada di lokasi server untuk melakukan konfigurasi maupun pemeliharaan karena tersedia berbagai teknologi yang memungkinkan akses jarak jauh secara aman. Salah satu teknologi tersebut adalah **Secure Shell (SSH)**, yaitu protokol yang menyediakan layanan komunikasi terenkripsi sehingga proses pertukaran data dan administrasi server terlindungi dari risiko penyadapan maupun akses yang tidak sah.

Salah satu platform virtualisasi yang banyak dimanfaatkan dalam lingkungan pendidikan maupun industri adalah **Proxmox Virtual Environment (Proxmox VE)**. Platform ini memungkinkan pengguna membuat, mengelola, dan memantau mesin virtual serta container melalui antarmuka web maupun terminal. Dukungan terhadap layanan SSH memberikan kemudahan bagi administrator untuk melakukan berbagai tugas administrasi, seperti konfigurasi sistem, pengelolaan layanan, serta pemantauan kondisi server secara lebih cepat dan efisien.

Dalam praktikum ini, koneksi SSH ke server Proxmox VE dilakukan menggunakan aplikasi **PuTTY** sebagai perangkat lunak klien SSH..

### 1.2 Tujuan

Adapun tujuan pelaksanaan praktikum ini adalah sebagai berikut.

1. Memahami konsep dan fungsi Secure Shell (SSH) sebagai protokol untuk akses jarak jauh ke server secara aman.
2. Menginstal dan mengonfigurasi Proxmox Virtual Environment (Proxmox VE) sebagai platform virtualisasi.
3. Membuat dan mengelola Virtual Machine (VM) serta Linux Container (LXC) pada Proxmox VE.
4. Melakukan konfigurasi jaringan pada Proxmox VE agar server dapat diakses melalui jaringan.

5. Mengimplementasikan SSH menggunakan aplikasi PuTTY untuk melakukan administrasi server Proxmox VE dari jarak jauh.

### **1.3 Manfaat**

Adapun manfaat yang diperoleh dari pelaksanaan praktikum ini adalah sebagai berikut.

#### **a. Bagi Mahasiswa**

Pelaksanaan praktikum ini memberikan kesempatan kepada mahasiswa untuk mengembangkan pengetahuan dan keterampilan dalam mengoperasikan server berbasis virtualisasi. Mahasiswa dapat mempelajari cara melakukan koneksi jarak jauh menggunakan protokol Secure Shell (SSH), memahami proses konfigurasi server Proxmox VE, serta mengelola sistem melalui aplikasi PuTTY. Pengalaman tersebut diharapkan dapat memperkuat kemampuan teknis yang diperlukan dalam administrasi sistem.

#### **b. Bagi Pembelajaran**

Praktikum ini mendukung proses pembelajaran dengan memberikan pengalaman langsung dalam mengimplementasikan materi yang telah dipelajari di kelas.

#### **c. Bagi Pengembangan Kompetensi**

Kegiatan praktikum ini berperan dalam meningkatkan kompetensi mahasiswa di bidang teknologi informasi, khususnya dalam konfigurasi server, pengelolaan Virtual Machine (VM) dan Linux Container (LXC), serta penerapan akses server yang aman menggunakan Secure Shell (SSH).

## **BAB II**

### **DASAR TEORI**

#### **2.1 Virtualisasi**

Menurut Stallings (2018), virtualisasi merupakan teknik yang digunakan untuk menciptakan lingkungan komputasi virtual dengan cara mengabstraksi sumber daya perangkat keras fisik sehingga dapat dimanfaatkan oleh lebih dari satu sistem operasi secara efisien. Konsep ini memberikan fleksibilitas dalam pengelolaan infrastruktur teknologi informasi sekaligus meningkatkan tingkat pemanfaatan perangkat keras.

Seiring berkembangnya teknologi informasi, virtualisasi telah menjadi salah satu komponen utama dalam pengelolaan pusat data (*data center*) dan layanan komputasi awan (*cloud computing*). Penerapan teknologi ini memberikan berbagai keuntungan, antara lain mempermudah proses penyediaan server, meningkatkan efisiensi penggunaan sumber daya, mempercepat proses pencadangan dan pemulihan sistem, serta mengurangi biaya investasi maupun operasional perangkat keras. Oleh karena itu, virtualisasi banyak digunakan oleh organisasi dan institusi sebagai solusi untuk membangun infrastruktur server yang lebih fleksibel, mudah dikelola, dan andal.

Secara teknis, virtualisasi dapat dibedakan menjadi beberapa jenis, yaitu virtualisasi platform (*platform virtualization*) yang menghasilkan Virtual Machine lengkap dengan sistem operasi tamu, virtualisasi berbasis sistem operasi (*operating system-level virtualization*) yang menghasilkan container, serta virtualisasi jaringan dan penyimpanan yang masing-masing berfokus pada abstraksi sumber daya jaringan dan media penyimpanan. Komponen utama yang memungkinkan virtualisasi platform berjalan adalah hypervisor, yang terbagi menjadi hypervisor tipe 1 (*bare-metal*) yang berjalan langsung di atas perangkat keras, dan hypervisor tipe 2 (*hosted*) yang berjalan di atas sistem operasi host.

Dari sisi keunggulan, virtualisasi memberikan efisiensi pemanfaatan perangkat keras, fleksibilitas dalam penyediaan sumber daya komputasi, serta mempermudah penerapan skema pemulihan bencana (*disaster recovery*). Meskipun demikian, virtualisasi juga memiliki keterbatasan, antara lain overhead kinerja akibat lapisan abstraksi tambahan, kompleksitas pengelolaan yang lebih tinggi

dibandingkan sistem fisik tunggal, serta risiko keamanan apabila terjadi kegagalan isolasi antar-lingkungan virtual (VM escape). Dalam konteks komputasi awan, virtualisasi menjadi fondasi utama bagi model layanan seperti Infrastructure as a Service (IaaS), yang memungkinkan penyedia layanan mengalokasikan sumber daya komputasi secara dinamis kepada banyak pengguna secara bersamaan.

## **2.2 Proxmox Virtual Environment (Proxmox VE)**

Proxmox Virtual Environment (Proxmox VE) merupakan platform virtualisasi berbasis Debian Linux yang bersifat sumber terbuka (open source). Berdasarkan dokumentasi resmi Proxmox Server Solutions GmbH (2024), Proxmox VE mengintegrasikan dua teknologi virtualisasi, yaitu Kernel-based Virtual Machine (KVM) untuk menjalankan Virtual Machine dan Linux Container (LXC) untuk menjalankan container.

Proxmox VE menyediakan antarmuka berbasis web yang memudahkan administrator dalam mengelola server virtual, penyimpanan, jaringan, pencadangan data, serta pemantauan kondisi sistem secara terpusat. Dukungan terhadap teknologi virtualisasi modern menjadikan Proxmox VE sebagai salah satu platform yang banyak digunakan pada lingkungan pendidikan, perusahaan, maupun penyedia layanan komputasi awan.

Proxmox VE pertama kali dirilis oleh Proxmox Server Solutions GmbH pada tahun 2008 sebagai solusi virtualisasi open source yang menggabungkan pengelolaan Virtual Machine dan container dalam satu platform terpadu. Sejak saat itu, Proxmox VE mengalami pengembangan berkelanjutan sehingga mendukung fitur-fitur lanjutan, seperti High Availability (HA) Cluster, Software-Defined Storage melalui Ceph, serta integrasi dengan sistem pencadangan Proxmox Backup Server.

Dari sisi cara kerja, Proxmox VE dibangun di atas kernel Linux yang telah dimodifikasi dengan dukungan modul KVM untuk virtualisasi perangkat keras penuh (full virtualization) serta dukungan LXC untuk virtualisasi ringan berbasis kernel bersama. Administrator dapat mengelola seluruh sumber daya, termasuk CPU, memori, penyimpanan, dan jaringan, melalui antarmuka web tanpa perlu berinteraksi langsung dengan baris perintah, meskipun akses melalui terminal dan SSH tetap tersedia untuk konfigurasi tingkat lanjut.

Kelebihan utama Proxmox VE terletak pada sifatnya yang open source sehingga dapat digunakan tanpa biaya lisensi, kemampuannya mengelola VM dan container dalam satu platform, serta dukungan komunitas yang aktif. Di sisi lain, Proxmox VE memiliki keterbatasan berupa kurva pembelajaran yang relatif lebih curam bagi pengguna baru dibandingkan platform virtualisasi berbasis antarmuka grafis sederhana, serta dukungan resmi berbayar (enterprise support) yang terpisah dari repositori komunitas. Dalam administrasi server modern, Proxmox VE banyak diadopsi oleh institusi pendidikan dan penyedia hosting berskala kecil hingga menengah karena efisiensi biaya dan fleksibilitas pengelolaannya, sekaligus menjadi dasar pembelajaran praktis mengenai konsep komputasi awan privat (private cloud).

### **2.3 Secure Shell (SSH)**

Secure Shell (SSH) merupakan protokol jaringan yang digunakan untuk melakukan komunikasi dan administrasi server secara aman melalui jaringan. Menurut Barrett, Silverman, dan Byrnes (2005), SSH dirancang untuk menggantikan protokol Telnet yang mengirimkan data dalam bentuk teks biasa (plain text), sehingga lebih rentan terhadap penyadapan.

SSH bekerja dengan menerapkan mekanisme autentikasi dan enkripsi terhadap seluruh data yang dikirimkan antara klien dan server. Dengan demikian, informasi seperti nama pengguna, kata sandi, maupun perintah yang dijalankan akan terlindungi dari ancaman penyadapan selama proses komunikasi berlangsung. Selain digunakan untuk akses terminal, SSH juga mendukung transfer berkas secara aman melalui protokol SCP (Secure Copy Protocol) maupun SFTP (SSH File Transfer Protocol).

Secara historis, SSH pertama kali dikembangkan oleh Tatu Ylönen pada tahun 1995 setelah terjadinya insiden penyadapan kata sandi pada jaringan Universitas Teknologi Helsinki. Protokol tersebut kemudian distandardisasi oleh Internet Engineering Task Force (IETF) menjadi SSH-2 melalui serangkaian dokumen Request for Comments (RFC) pada awal tahun 2000-an, sehingga menjadi protokol akses jarak jauh yang paling banyak digunakan pada sistem berbasis Unix dan Linux hingga saat ini.

Mekanisme kerja SSH secara umum terdiri atas tiga lapisan, yaitu lapisan transport yang bertanggung jawab atas enkripsi kanal komunikasi menggunakan algoritma kriptografi simetris seperti AES, lapisan autentikasi yang memverifikasi identitas pengguna menggunakan kata sandi atau pasangan kunci publik-privat, dan lapisan koneksi yang mengatur sesi terminal, penerusan port (port forwarding), maupun transfer berkas dalam satu kanal terenkripsi. Proses pertukaran kunci (key exchange) pada tahap awal koneksi memungkinkan klien dan server menyepakati kunci sesi tanpa mengirimkannya secara langsung melalui jaringan, sehingga meminimalkan risiko peretasan.

Keunggulan SSH terletak pada tingkat keamanannya yang tinggi, dukungan autentikasi berlapis, serta fleksibilitas penggunaannya untuk berbagai keperluan administrasi jarak jauh. Meskipun demikian, keamanan SSH tetap bergantung pada konfigurasi yang diterapkan administrator, seperti penggunaan kata sandi yang lemah atau login root yang dibiarkan aktif, sehingga tetap berpotensi disalahgunakan apabila tidak dikonfigurasi dengan baik. Dalam praktik administrasi server modern, SSH menjadi komponen fundamental yang mendukung otomatisasi melalui perangkat seperti Ansible, pengelolaan infrastruktur berbasis cloud, serta praktik keamanan jaringan seperti tunneling dan bastion host untuk membatasi titik akses ke sistem internal.

## **2.4 PuTTY**

PuTTY merupakan aplikasi klien SSH yang dikembangkan untuk sistem operasi Windows dan digunakan sebagai media akses jarak jauh ke server berbasis Linux maupun Unix. Menurut dokumentasi resmi PuTTY (2024), aplikasi ini mendukung berbagai protokol komunikasi, seperti SSH, Telnet, Serial, dan Rlogin.

Dalam administrasi server, PuTTY banyak digunakan karena memiliki antarmuka yang sederhana, mudah dikonfigurasi, serta mampu melakukan autentikasi menggunakan kata sandi maupun pasangan kunci publik (public key authentication). Pada praktikum ini, PuTTY dimanfaatkan sebagai aplikasi untuk menghubungkan komputer klien dengan server Proxmox VE menggunakan protokol SSH.

PuTTY pertama kali dikembangkan oleh Simon Tatham pada akhir dekade 1990-an sebagai aplikasi terminal emulator gratis dan bersifat open source untuk

pengguna Windows yang membutuhkan akses ke server Unix maupun Linux. Seiring berjalannya waktu, PuTTY berkembang menjadi salah satu aplikasi klien SSH yang paling populer karena ukurannya yang ringan, tidak memerlukan proses instalasi (portable), serta terus dipelihara secara aktif oleh komunitas pengembang.

Dari sisi fungsionalitas, PuTTY tidak hanya berfungsi sebagai klien SSH, tetapi juga menyediakan modul pendukung lain, seperti PuTTYgen untuk menghasilkan pasangan kunci publik dan privat, Pageant sebagai agen penyimpan kunci SSH, serta PSCP dan PSFTP untuk transfer berkas melalui baris perintah. Kombinasi modul tersebut menjadikan PuTTY sebagai satu paket perangkat lunak yang cukup lengkap untuk kebutuhan administrasi server jarak jauh tanpa memerlukan aplikasi tambahan.

Kelebihan PuTTY antara lain ukuran aplikasi yang kecil, kompatibilitas luas dengan berbagai versi Windows, serta kemampuan menyimpan profil konfigurasi sesi untuk mempercepat proses koneksi berikutnya. Kekurangannya, PuTTY memiliki antarmuka berbasis teks yang kurang ramah bagi pengguna awam serta minim fitur manajemen sesi tingkat lanjut dibandingkan aplikasi klien SSH modern seperti MobaXterm atau Termius. Dalam praktik administrasi server, PuTTY tetap relevan digunakan karena kesederhanaannya, khususnya pada lingkungan pembelajaran dan pengujian konfigurasi jaringan skala kecil seperti pada praktikum ini.

## 2.5 Oracle VirtualBox

**Oracle VirtualBox** adalah perangkat lunak virtualisasi yang berfungsi sebagai *hypervisor* tipe 2, sehingga memungkinkan pengguna membuat dan menjalankan beberapa mesin virtual (*Virtual Machine*) pada satu komputer fisik. Melalui VirtualBox, berbagai sistem operasi dapat dijalankan secara bersamaan tanpa mengganggu sistem operasi utama (*host*). Menurut Oracle Corporation (2024), VirtualBox mendukung beragam sistem operasi tamu (*guest operating system*), di antaranya Windows, Linux, Solaris, dan BSD.

Pada praktikum ini, Oracle VirtualBox versi 6.1.50 digunakan sebagai host hypervisor untuk menjalankan Proxmox VE sehingga seluruh proses instalasi dan konfigurasi server dapat dilakukan tanpa memerlukan perangkat keras tambahan.

VirtualBox awalnya dikembangkan oleh perusahaan Innotek GmbH pada tahun 2007 sebelum diakuisisi oleh Sun Microsystems, dan selanjutnya dikelola oleh Oracle Corporation setelah akuisisi Sun Microsystems pada tahun 2010. Sebagai hypervisor tipe 2, VirtualBox berjalan di atas sistem operasi host yang sudah terpasang, sehingga proses instalasinya relatif sederhana dan cocok digunakan pada lingkungan pembelajaran maupun pengujian yang tidak memerlukan performa setara server produksi.

Cara kerja VirtualBox memanfaatkan fitur virtualisasi perangkat keras (hardware-assisted virtualization) yang terdapat pada prosesor modern, seperti Intel VT-x dan AMD-V, untuk menjalankan sistem operasi tamu secara terisolasi dari sistem host. VirtualBox juga menyediakan fitur jaringan virtual yang fleksibel, seperti NAT, Bridged Adapter, dan Host-only Adapter, sehingga memungkinkan konfigurasi topologi jaringan sesuai kebutuhan, termasuk konfigurasi Port Forwarding yang digunakan pada praktikum ini untuk meneruskan lalu lintas dari komputer host menuju layanan yang berjalan pada mesin virtual.

Kelebihan VirtualBox terletak pada sifatnya yang gratis, mendukung banyak sistem operasi host dan tamu, serta memiliki antarmuka grafis yang memudahkan konfigurasi bagi pengguna pemula. Kekurangannya, performa VirtualBox umumnya berada di bawah hypervisor bare-metal seperti VMware ESXi atau Proxmox VE itu sendiri apabila digunakan untuk beban kerja produksi berskala besar, karena adanya overhead dari sistem operasi host. Dalam konteks pembelajaran administrasi server dan virtualisasi bertingkat (nested virtualization), VirtualBox tetap menjadi pilihan yang relevan karena memungkinkan simulasi lingkungan server sungguhan, seperti Proxmox VE, tanpa memerlukan perangkat keras server fisik.

## **2.6 Virtual Machine (VM)**

Virtual Machine (VM) merupakan representasi virtual dari sebuah komputer yang memiliki prosesor, memori, media penyimpanan, dan antarmuka jaringan virtual. Menurut VMware (2023), setiap VM bekerja secara independen layaknya komputer fisik sehingga dapat menjalankan sistem operasi dan aplikasi secara terpisah.

Pada Proxmox VE, VM dijalankan menggunakan teknologi Kernel-based Virtual Machine (KVM) yang memanfaatkan kemampuan virtualisasi pada prosesor modern. Teknologi ini memungkinkan sistem operasi tamu memperoleh performa yang mendekati sistem fisik.

Secara arsitektural, setiap VM memiliki kernel sistem operasi tersendiri yang berjalan secara independen dari kernel sistem host maupun VM lainnya, sehingga tingkat isolasinya sangat tinggi. Komponen virtual yang dimiliki setiap VM, seperti vCPU (virtual CPU), memori virtual, disk virtual, dan antarmuka jaringan virtual, dikelola oleh hypervisor dan dipetakan ke sumber daya fisik yang tersedia pada server host melalui mekanisme penjadwalan (scheduling).

Kelebihan utama VM adalah fleksibilitasnya dalam menjalankan sistem operasi tamu yang berbeda dari sistem operasi host, misalnya menjalankan Windows Server di atas host Linux, serta tingkat isolasi keamanan yang lebih kuat karena setiap VM memiliki kernel terpisah. Kekurangannya, VM membutuhkan alokasi sumber daya yang lebih besar dan waktu booting yang lebih lama dibandingkan container, karena setiap VM harus memuat sistem operasi secara utuh. Dalam administrasi server, pemilihan antara VM dan container umumnya didasarkan pada kebutuhan isolasi, jenis sistem operasi yang dijalankan, serta efisiensi sumber daya yang diinginkan.

## **2.7 Linux Container (LXC)**

Linux Container (LXC) merupakan teknologi virtualisasi berbasis sistem operasi (operating system-level virtualization) yang memungkinkan beberapa lingkungan Linux berjalan pada satu kernel yang sama. Berbeda dengan Virtual Machine, container tidak memerlukan kernel tersendiri sehingga penggunaan memori dan prosesor menjadi lebih efisien.

Menurut dokumentasi Linux Containers Project (2024), LXC memberikan isolasi terhadap proses, sistem berkas, jaringan, dan pengguna sehingga setiap container dapat berjalan secara mandiri meskipun berbagi kernel dengan sistem operasi utama. Oleh karena itu, LXC banyak digunakan untuk menjalankan layanan yang membutuhkan waktu booting cepat dan konsumsi sumber daya yang rendah.

Teknologi LXC memanfaatkan fitur kernel Linux, yaitu namespace dan control groups (cgroups). Namespace bertugas mengisolasi tampilan proses,

jaringan, dan sistem berkas antar-container, sedangkan cgroups mengatur pembatasan dan alokasi sumber daya, seperti CPU dan memori, untuk masing-masing container. Kombinasi kedua mekanisme tersebut memungkinkan beberapa container berjalan secara independen pada satu kernel host tanpa saling mengganggu.

Dibandingkan dengan Virtual Machine, Linux Container memiliki keunggulan berupa waktu booting yang jauh lebih singkat, konsumsi sumber daya yang lebih hemat, serta kepadatan (density) container yang dapat dijalankan dalam satu server fisik jauh lebih tinggi. Namun demikian, karena seluruh container berbagi kernel yang sama dengan host, tingkat isolasi keamanannya secara umum dianggap lebih rendah dibandingkan VM, sehingga LXC kurang direkomendasikan untuk menjalankan beban kerja yang saling tidak dipercaya (multi-tenant yang sangat sensitif) tanpa lapisan keamanan tambahan. Dalam praktikum ini, penggunaan LXC dipilih karena kebutuhan sumber daya yang lebih ringan dan proses konfigurasi yang lebih cepat dibandingkan VM, sehingga sesuai untuk simulasi implementasi SSH pada lingkungan terbatas.

## **2.8 Konfigurasi Jaringan pada Proxmox VE**

Konfigurasi jaringan merupakan tahapan penting dalam implementasi virtualisasi karena menentukan kemampuan komunikasi antara server, Virtual Machine, maupun Linux Container. Pada Proxmox VE, konfigurasi jaringan umumnya dilakukan menggunakan Linux Bridge, yaitu mekanisme yang memungkinkan perangkat virtual terhubung ke jaringan fisik melalui antarmuka jaringan server.

Melalui konfigurasi tersebut, administrator dapat mengakses server menggunakan protokol SSH, melakukan pengelolaan mesin virtual, serta menghubungkan layanan yang berjalan pada VM maupun container ke jaringan lokal maupun internet. Konfigurasi jaringan yang tepat akan mendukung keamanan, stabilitas, dan ketersediaan layanan dalam lingkungan virtualisasi.

Selain Linux Bridge, Proxmox VE juga mendukung skema jaringan lanjutan, seperti Open vSwitch untuk kebutuhan jaringan yang lebih kompleks, VLAN (Virtual Local Area Network) untuk melakukan segmentasi lalu lintas jaringan, serta bonding antarmuka jaringan fisik untuk meningkatkan ketersediaan

dan throughput koneksi. Pada lingkungan pengujian menggunakan VirtualBox seperti praktikum ini, konfigurasi jaringan disederhanakan melalui mekanisme Port Forwarding pada adaptor NAT, yang memetakan port tertentu pada komputer host ke port yang sesuai pada mesin virtual Proxmox VE.

Konfigurasi jaringan yang tepat memiliki keterkaitan erat dengan aspek keamanan siber, karena kesalahan pengaturan, seperti membuka port yang tidak diperlukan atau tidak menerapkan pembatasan akses, dapat memperbesar permukaan serangan (attack surface) terhadap server. Oleh karena itu, penerapan prinsip keamanan berlapis, termasuk penggunaan firewall, pembatasan alamat IP yang diperbolehkan mengakses layanan SSH, serta segmentasi jaringan antara lingkungan produksi dan pengujian, menjadi bagian penting dalam pengelolaan jaringan pada platform virtualisasi seperti Proxmox VE.

## BAB III

### PRAKTIKUM

#### 3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam pelaksanaan praktikum adalah sebagai berikut.

##### a. Perangkat Keras

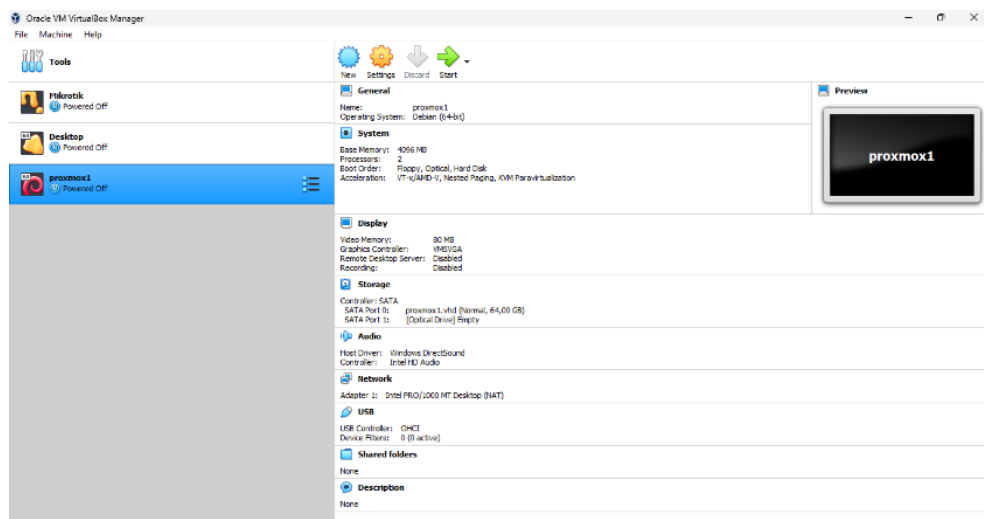
- Satu unit komputer atau laptop.
- Mouse dan keyboard.
- Koneksi jaringan lokal (LAN) atau jaringan virtual.

##### b. Perangkat Lunak

- Oracle VirtualBox versi 6.1.50.
- PuTTY.
- Web browser.
- File ISO Proxmox VE 6.4-1.

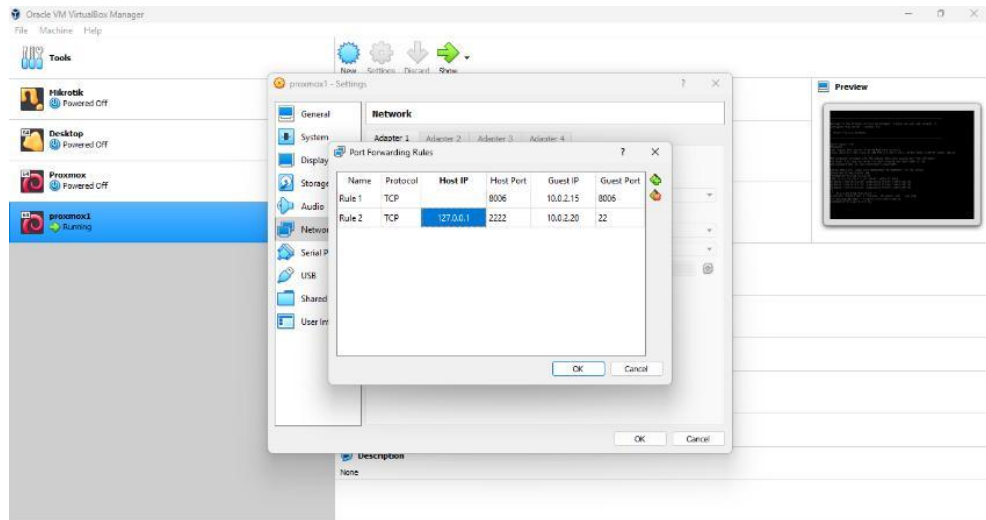
#### 3.2 Prosedur Praktikum

1. Membuka aplikasi Oracle VM VirtualBox versi 6.1.50, kemudian membuat mesin virtual (Virtual Machine) baru dengan nama proxmox1. Selanjutnya dilakukan konfigurasi memori, prosesor, media penyimpanan, dan jaringan sebagai persiapan instalasi Proxmox VE 6.4-1.



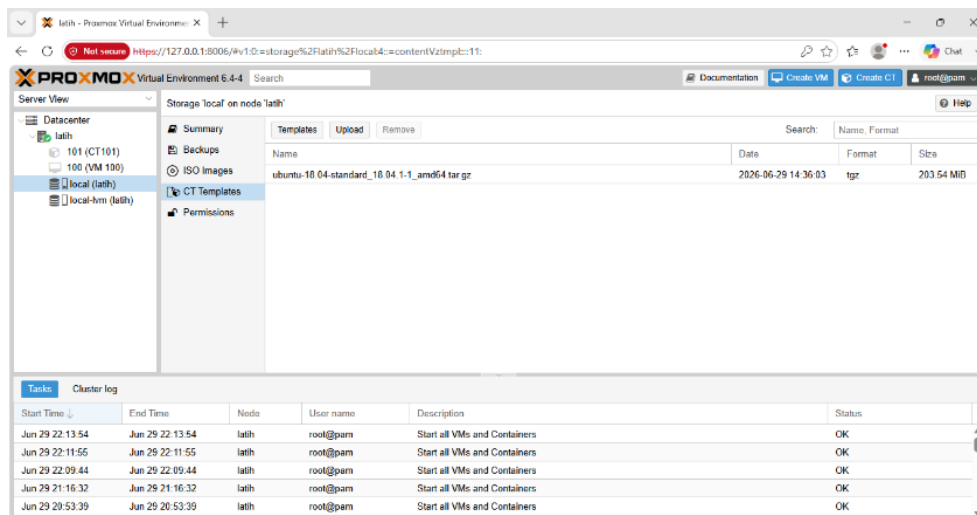
Gambar 3.1 Pembuatan mesin virtual (Virtual Machine) proxmox1 pada Oracle VM VirtualBox.

2. Melakukan konfigurasi **Port Forwarding** pada adaptor jaringan VirtualBox untuk menghubungkan komputer *host* dengan mesin virtual. Konfigurasi ini dilakukan agar layanan **Proxmox VE** melalui port **8006** dan layanan **SSH** melalui port **22** dapat diakses dari komputer *host*.



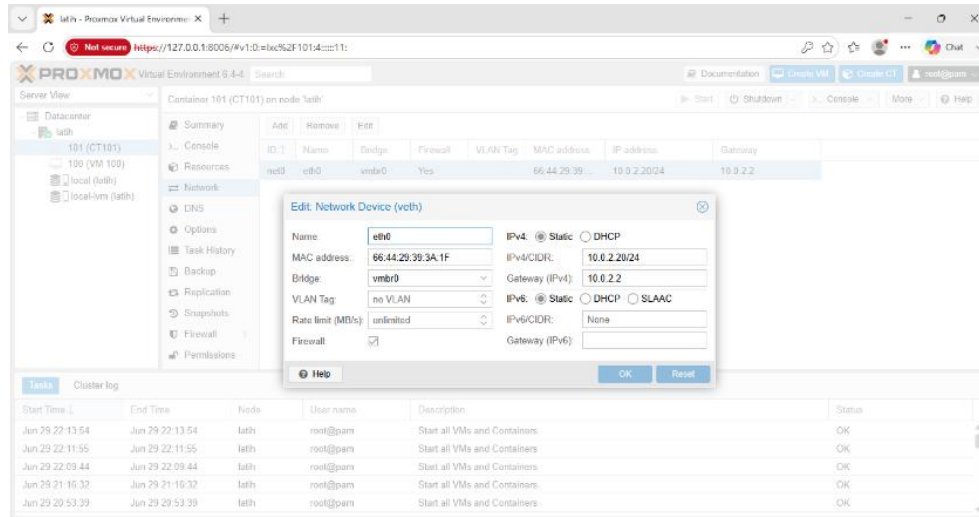
Gambar 3.2 Konfigurasi Port Forwarding pada Oracle VM VirtualBox untuk akses Proxmox VE dan SSH.

3. Mengakses antarmuka **Proxmox VE** melalui *web browser* menggunakan alamat **<https://127.0.0.1:8006>**. Selanjutnya memastikan bahwa **CT Template Ubuntu 18.04** telah tersedia sebagai persiapan pembuatan **Linux Container (LXC)**.



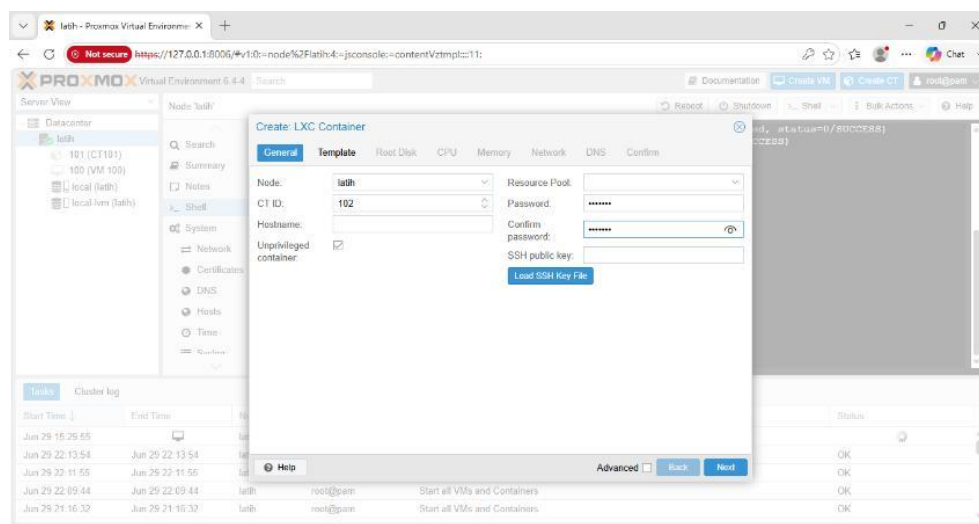
Gambar 3.3 Tampilan antarmuka Proxmox VE pada menu CT Templates.

4. Melakukan konfigurasi jaringan pada **Linux Container (LXC)** dengan menetapkan alamat **IP statis**, **gateway**, dan **bridge (vmbro0)** agar container dapat terhubung ke jaringan dan diakses melalui SSH.



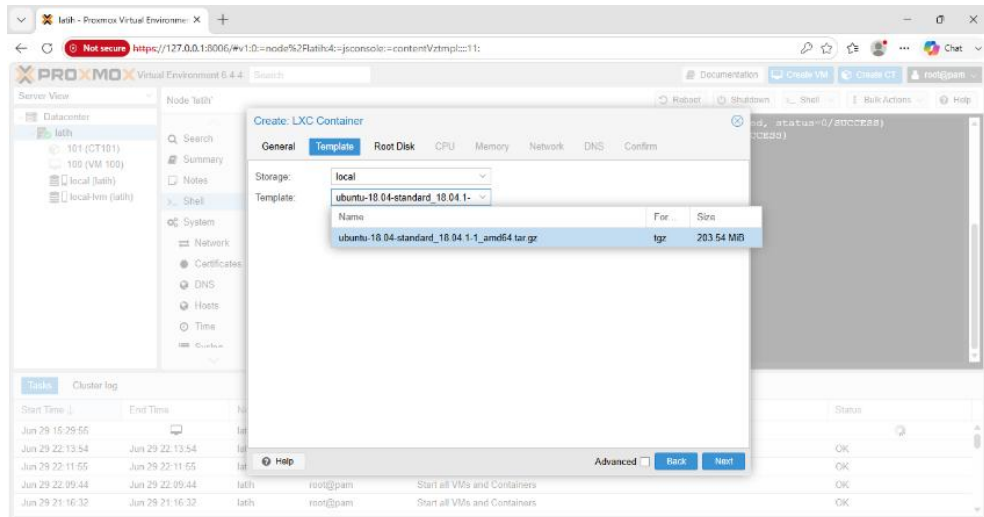
*Gambar 3.4 Konfigurasi jaringan pada Linux Container (LXC).*

5. Membuat **Linux Container (LXC)** baru dengan memilih menu **Create CT**, kemudian mengisi identitas container, seperti **CT ID**, **hostname**, dan **kata sandi** sebagai langkah awal pembuatan container.



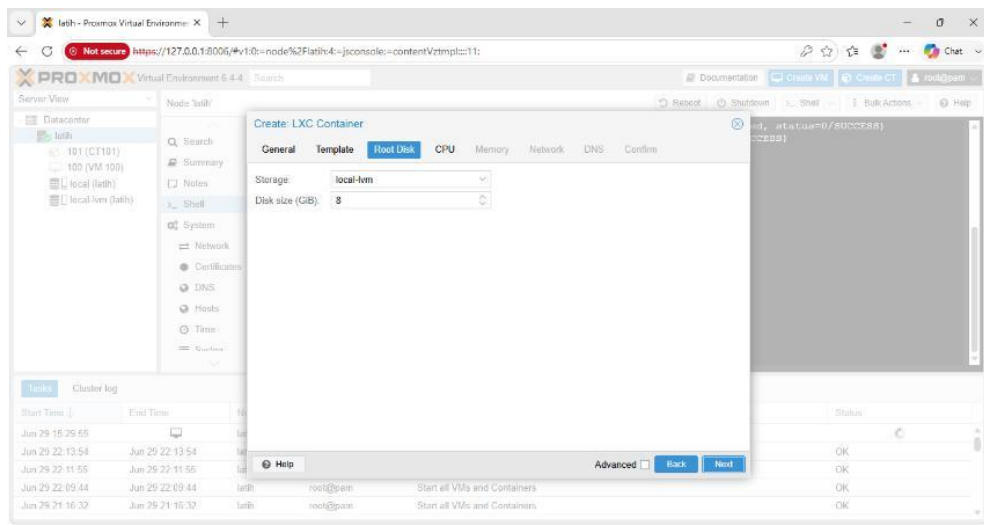
*Gambar 3.5 Pengisian identitas Linux Container (LXC) pada menu Create CT di Proxmox VE.*

6. Memilih **template sistem operasi Ubuntu 18.04** yang telah diunggah pada penyimpanan **local** sebagai dasar pembuatan **Linux Container (LXC)**.



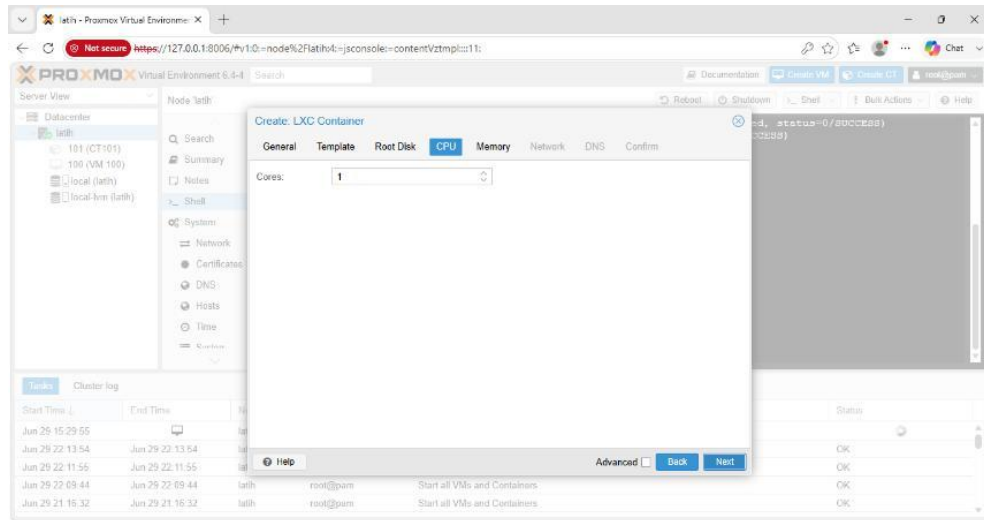
Gambar 3.6 Pemilihan template Ubuntu 18.04 pada proses pembuatan Linux Container (LXC) di Proxmox VE.

7. Menentukan lokasi penyimpanan (*storage*) pada **local-lvm** serta mengatur kapasitas **Root Disk** sebesar **8 GiB** untuk **Linux Container (LXC)**.



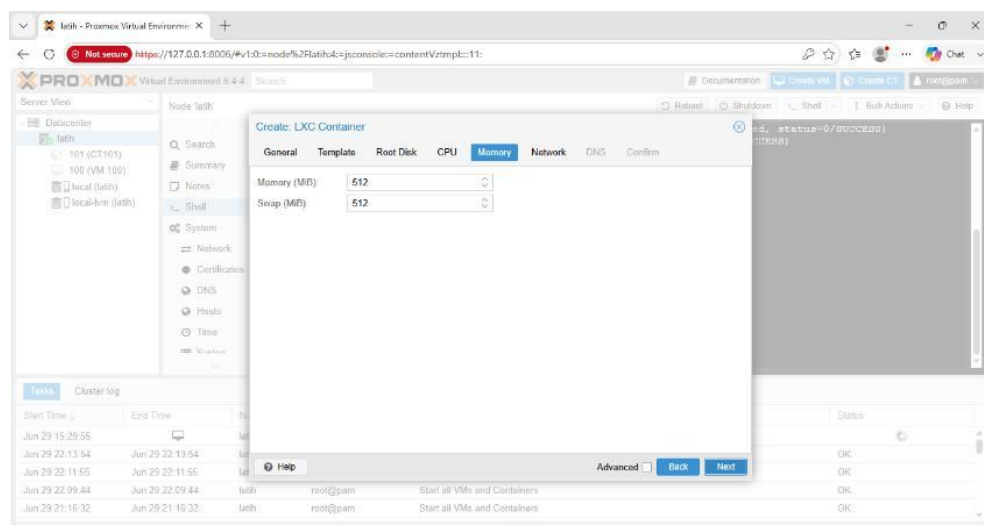
Gambar 3.7 Konfigurasi Root Disk pada proses pembuatan Linux Container (LXC) di Proxmox VE.

8. Menentukan jumlah **CPU (Core)** yang akan dialokasikan untuk **Linux Container (LXC)** sesuai dengan kebutuhan sistem.



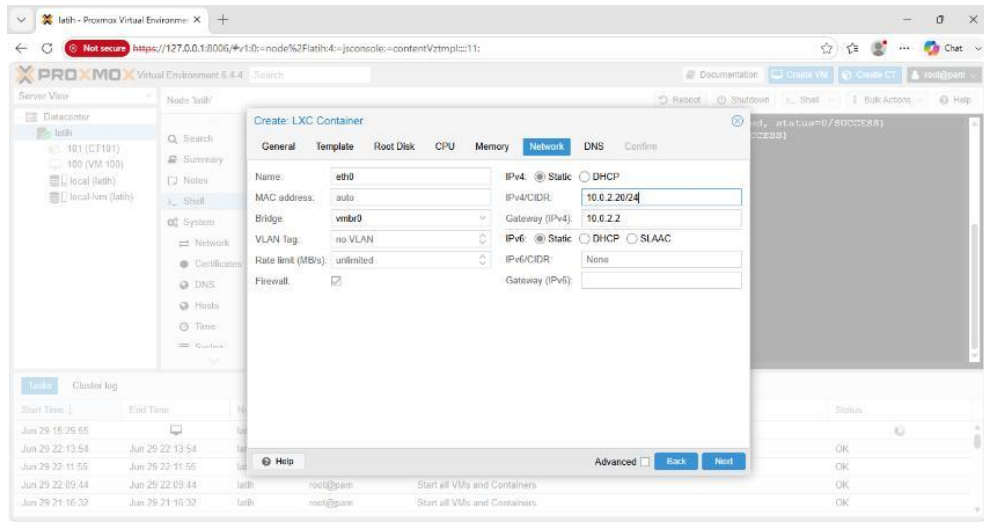
*Gambar 3.8 Konfigurasi CPU (Core) pada proses pembuatan Linux Container (LXC) di Proxmox VE.*

9. Menentukan kapasitas **memori (RAM)** dan **Swap** yang akan dialokasikan untuk **Linux Container (LXC)** sesuai dengan kebutuhan sistem.



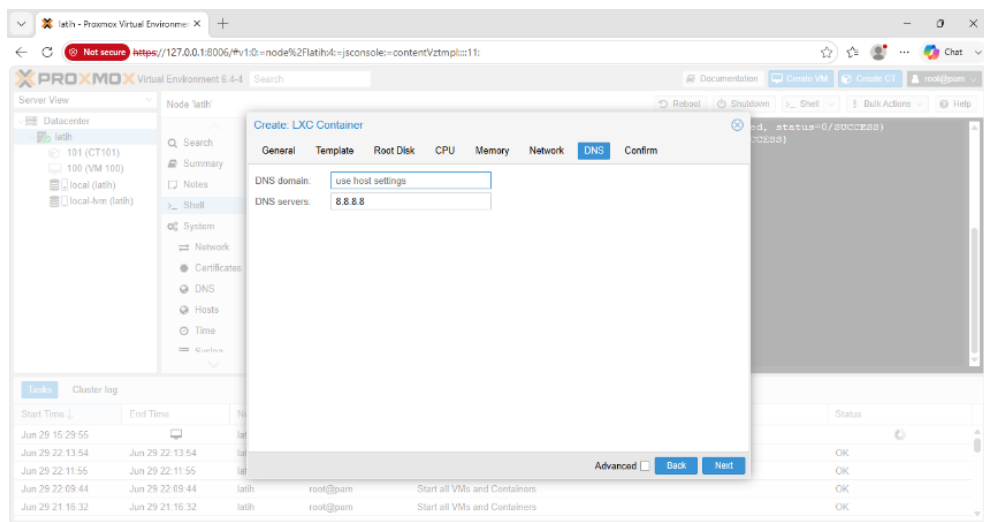
*Gambar 3.9 Konfigurasi memori (RAM) dan Swap pada proses pembuatan Linux Container (LXC) di Proxmox VE.*

10. Melakukan konfigurasi jaringan dengan menentukan **bridge (vbr0)**, **alamat IP statis**, dan **gateway** agar **Linux Container (LXC)** dapat terhubung ke jaringan.



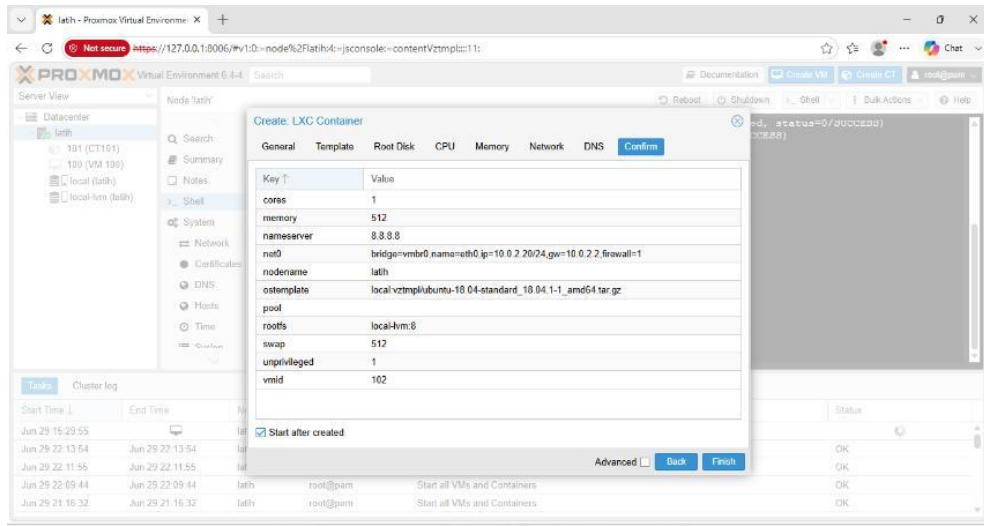
Gambar 3.10 Konfigurasi jaringan pada proses pembuatan Linux Container (LXC) di Proxmox VE.

11. Melakukan konfigurasi **DNS** dengan menggunakan pengaturan bawaan (*Use Host Settings*) serta menentukan **DNS Server 8.8.8.8** agar **Linux Container (LXC)** dapat melakukan resolusi nama domain.



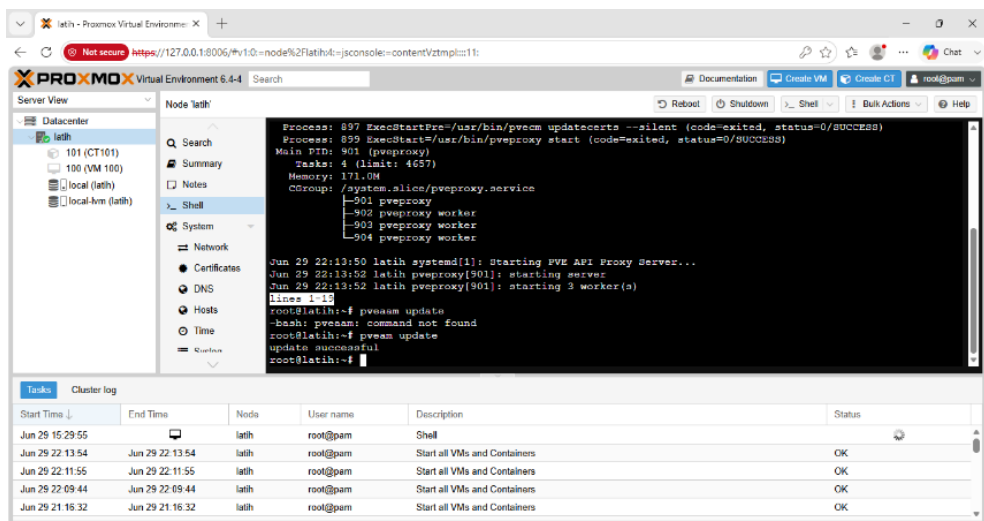
Gambar 3.11 Konfigurasi DNS pada proses pembuatan Linux Container (LXC) di Proxmox VE.

12. Memastikan seluruh konfigurasi **Linux Container (LXC)** telah sesuai, kemudian memilih tombol **Finish** untuk memulai proses pembuatan container.



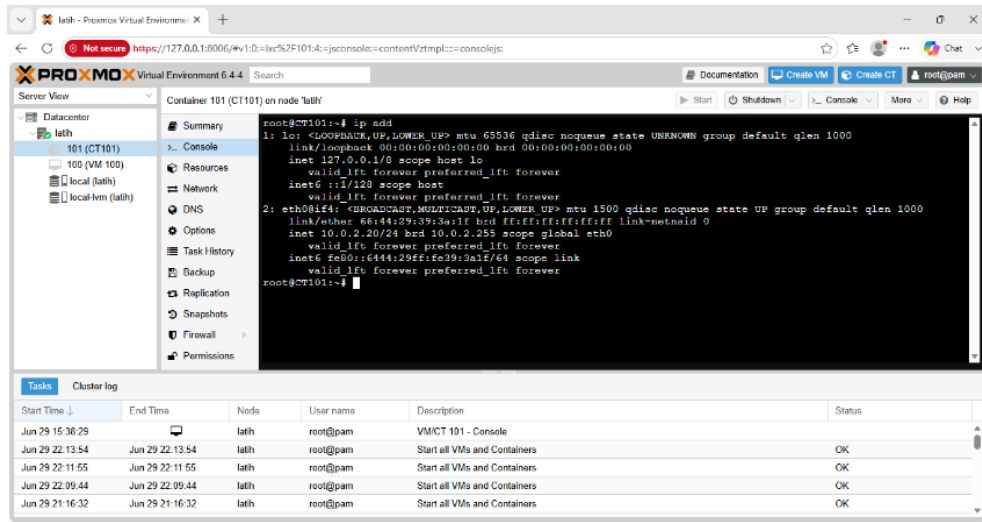
Gambar 3.12 Konfirmasi konfigurasi sebelum pembuatan Linux Container (LXC) di Proxmox VE.

13. Membuka menu **Shell** pada Proxmox VE, kemudian menjalankan perintah **pveam update** untuk memperbarui daftar template **Linux Container (LXC)** yang tersedia pada repositori Proxmox.



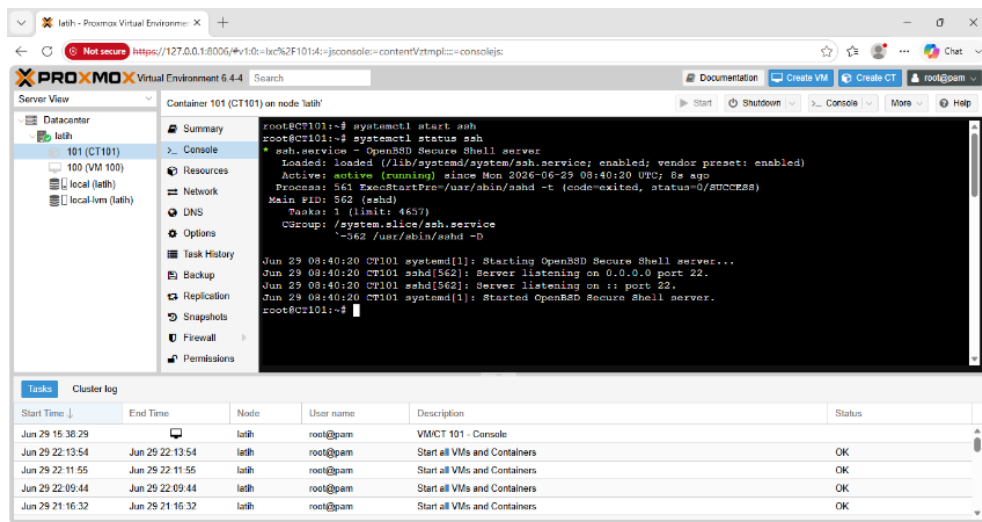
Gambar 3.13 Menjalankan perintah **pveam update** pada Shell Proxmox VE.

14. Membuka **Console** pada **Linux Container (CT101)**, kemudian menjalankan perintah **ip add** untuk memverifikasi konfigurasi jaringan dan memastikan alamat IP telah berhasil diterapkan.



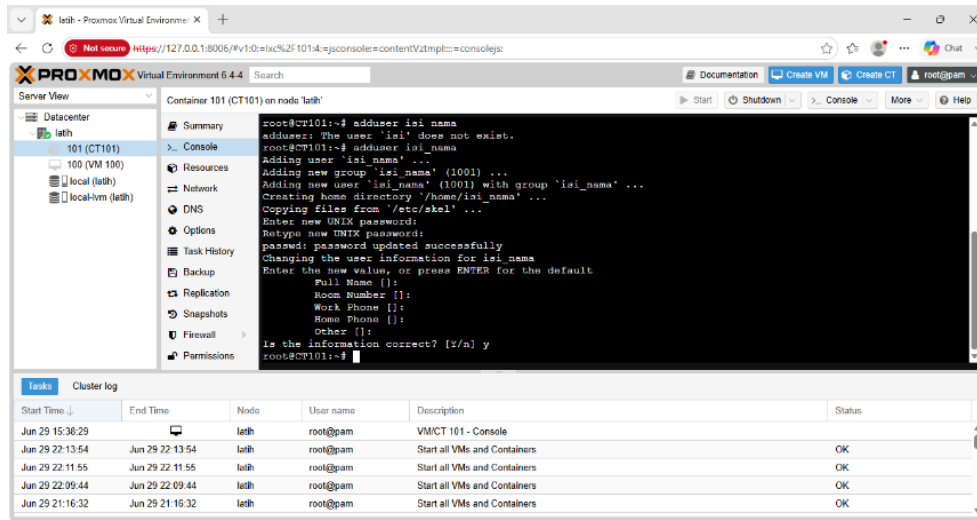
Gambar 3.14 Verifikasi konfigurasi jaringan menggunakan perintah `ip add` pada Linux Container (CT101).

- Menjalankan perintah `systemctl start ssh` untuk mengaktifkan layanan SSH, kemudian memverifikasi status layanan menggunakan perintah `systemctl status ssh` guna memastikan layanan SSH telah berjalan dengan baik.



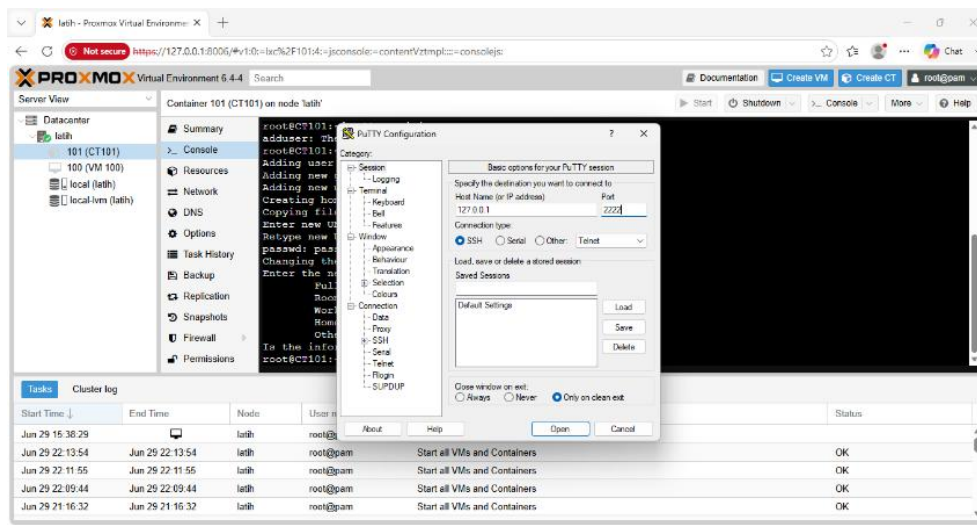
Gambar 3.15 Mengaktifkan dan memverifikasi layanan SSH pada Linux Container (CT101).

- Membuat akun pengguna baru pada **Linux Container (LXC)** menggunakan perintah `adduser`, kemudian mengatur kata sandi dan melengkapi informasi pengguna sebagai persiapan akses melalui SSH.



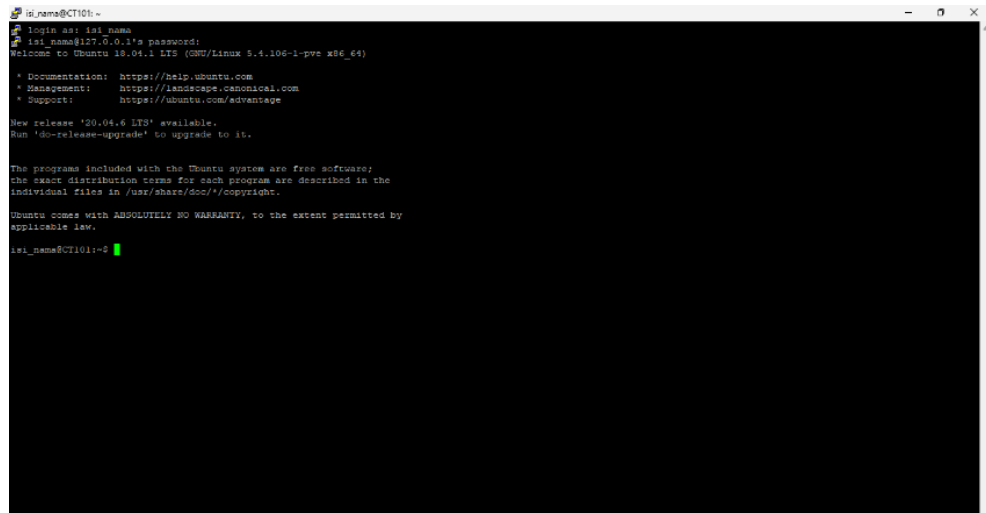
Gambar 3.16 Pembuatan akun pengguna baru pada Linux Container (CT101) menggunakan perintah `adduser`.

17. Membuka aplikasi **PuTTY**, kemudian mengisi **Host Name** dengan alamat **127.0.0.1**, **Port** dengan **2222**, memilih jenis koneksi **SSH**, lalu menekan tombol **Open** untuk menghubungkan komputer *host* dengan **Linux Container**.



Gambar 3.17 Konfigurasi koneksi SSH menggunakan aplikasi PuTTY.

18. Melakukan **login** ke **Linux Container (CT101)** melalui aplikasi **PuTTY** menggunakan akun pengguna yang telah dibuat sebelumnya. Proses login berhasil ditandai dengan munculnya *prompt* terminal pada sistem Ubuntu.



```
isi_nama@CT101:~$ login as: isi_nama
isi_nama@127.0.0.1's password:
Welcome to Ubuntu 18.04.1 LTS (GNU/Linux 5.4.106-1-pve x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

isi_nama@CT101:~$
```

*Gambar 3.18 Hasil login ke Linux Container (CT101) menggunakan SSH melalui aplikasi PuTTY.*

## **BAB IV**

### **HASIL DAN PEMBAHASAN**

#### **4.1 Hasil Praktikum**

Berdasarkan praktikum yang telah dilaksanakan, seluruh tahapan implementasi Secure Shell (SSH) pada server Proxmox VE menggunakan PuTTY berhasil dilakukan dengan baik. Proses diawali dengan pembuatan mesin virtual pada Oracle VM VirtualBox versi 6.1.50 sebagai media instalasi Proxmox VE versi 6.4-1. Setelah proses instalasi selesai, Proxmox VE berhasil diakses melalui antarmuka berbasis web menggunakan peramban (web browser).

Selanjutnya dilakukan konfigurasi Port Forwarding pada VirtualBox agar layanan Proxmox VE dan SSH dapat diakses dari komputer host. Setelah konfigurasi jaringan berhasil dilakukan, proses pembuatan Linux Container (LXC) dapat dilaksanakan dengan menentukan identitas container, template sistem operasi Ubuntu 18.04, kapasitas penyimpanan, prosesor, memori, serta konfigurasi jaringan dan DNS.

Tahap berikutnya adalah mengaktifkan layanan SSH pada Linux Container menggunakan perintah `systemctl start ssh` dan memverifikasi status layanan menggunakan perintah `systemctl status ssh`. Hasil pengujian menunjukkan bahwa layanan SSH berstatus active (running), yang menandakan bahwa layanan telah berjalan dengan baik.

Sebagai tahap akhir, dilakukan pengujian koneksi menggunakan aplikasi PuTTY dengan memanfaatkan alamat 127.0.0.1 dan port 2222 sesuai konfigurasi Port Forwarding. Hasil pengujian menunjukkan bahwa proses autentikasi berhasil dilakukan menggunakan akun pengguna yang telah dibuat sebelumnya. Setelah berhasil masuk ke sistem Ubuntu melalui terminal PuTTY, pengguna dapat menjalankan perintah pada Linux Container dari jarak jauh. Hal tersebut menunjukkan bahwa implementasi SSH pada server Proxmox VE telah berhasil diterapkan.

#### **4.2 Pembahasan**

Implementasi Secure Shell (SSH) pada praktikum ini bertujuan untuk menyediakan akses jarak jauh yang aman ke server berbasis Linux. SSH bekerja

dengan mengenkripsi komunikasi antara komputer klien dan server sehingga data yang dikirimkan, termasuk nama pengguna, kata sandi, dan perintah yang dijalankan, terlindungi dari penyadapan selama proses komunikasi berlangsung.

Pada praktikum ini, Proxmox VE berperan sebagai platform virtualisasi yang digunakan untuk mengelola Linux Container (LXC). Agar layanan SSH dapat diakses dari komputer host, dilakukan konfigurasi Port Forwarding pada Oracle VM VirtualBox dengan meneruskan Host Port 2222 ke Guest Port 22. Konfigurasi tersebut memungkinkan aplikasi PuTTY melakukan koneksi ke layanan SSH yang berjalan pada Linux Container.

Keberhasilan implementasi SSH dibuktikan dengan status layanan active (running) serta keberhasilan proses login melalui aplikasi PuTTY. Setelah koneksi berhasil dibuat, pengguna dapat melakukan administrasi sistem melalui terminal tanpa harus mengakses console Proxmox VE secara langsung. Dengan demikian, proses pengelolaan server menjadi lebih mudah, fleksibel, dan efisien.

Dalam konteks administrasi server, SSH berperan sebagai kanal utama yang memungkinkan administrator melakukan konfigurasi sistem, pemantauan layanan, serta pemeliharaan berkala tanpa harus berada secara fisik di lokasi server. Pemilihan PuTTY sebagai aplikasi klien SSH pada praktikum ini didasarkan pada kesederhanaan penggunaannya, ukuran aplikasi yang ringan, serta kemampuannya menyimpan konfigurasi sesi sehingga proses koneksi berulang ke Linux Container dapat dilakukan dengan lebih cepat dibandingkan harus mengetikkan parameter koneksi setiap kali mengakses server.

Konfigurasi Port Forwarding yang diterapkan pada praktikum ini, yaitu meneruskan Host Port 2222 menuju Guest Port 22, memungkinkan lalu lintas SSH dari komputer host diarahkan menuju layanan SSH yang berjalan pada Linux Container meskipun container tersebut berada dalam jaringan virtual yang terisolasi dari jaringan fisik. Pemilihan port non-standar (2222) alih-alih port default (22) pada sisi host juga mencerminkan praktik umum untuk menghindari konflik dengan layanan lain serta sedikit mengurangi eksposur terhadap pemindaian port otomatis yang umum menyoroti port 22.

Penggunaan Linux Container pada praktikum ini, dibandingkan dengan Virtual Machine, memberikan keuntungan berupa waktu pembuatan dan booting

yang jauh lebih cepat serta konsumsi sumber daya memori dan penyimpanan yang lebih rendah, sehingga sesuai dengan keterbatasan sumber daya pada lingkungan pengujian berbasis VirtualBox. Meskipun demikian, perlu disadari bahwa tingkat isolasi LXC yang berbagi kernel dengan host secara teoritis lebih rentan dibandingkan VM apabila diterapkan pada lingkungan produksi yang menangani banyak pengguna dengan tingkat kepercayaan berbeda.

Mekanisme autentikasi yang digunakan pada praktikum ini adalah autentikasi berbasis kata sandi (password-based authentication), di mana pengguna memasukkan nama pengguna dan kata sandi yang telah dibuat menggunakan perintah `adduser` sebelum diberikan akses ke terminal Linux Container. Meskipun metode ini sederhana dan mudah diterapkan, keamanannya tetap bergantung pada kekuatan kata sandi yang digunakan, sehingga rentan terhadap serangan brute force apabila tidak dilengkapi mekanisme pembatasan percobaan login (`fail2ban`) atau tidak digantikan dengan autentikasi berbasis kunci publik yang jauh lebih tahan terhadap upaya penembakan kata sandi.

Enkripsi pada komunikasi SSH menjadi aspek yang sangat penting karena seluruh data yang dipertukarkan antara PuTTY dan Linux Container, termasuk kredensial login dan perintah yang dijalankan, terlindungi dari kemungkinan penyadapan oleh pihak yang tidak berwenang selama berada dalam satu jaringan yang sama. Tanpa enkripsi tersebut, sebagaimana yang terjadi pada protokol Telnet, seluruh informasi sensitif akan dikirimkan dalam bentuk teks biasa yang dapat dibaca secara langsung apabila lalu lintas jaringan berhasil disadap.

Selama pelaksanaan praktikum, beberapa kendala teknis ditemukan, antara lain kegagalan koneksi PuTTY akibat konfigurasi Port Forwarding yang belum sesuai, serta layanan SSH yang belum aktif pada Linux Container sehingga permintaan koneksi ditolak. Kendala tersebut diatasi dengan memeriksa kembali kesesuaian pemetaan Host Port dan Guest Port pada VirtualBox, serta memastikan status layanan SSH melalui perintah `systemctl status ssh` sebelum melakukan percobaan koneksi berikutnya. Pengalaman tersebut menegaskan pentingnya verifikasi bertahap (troubleshooting sistematis) dalam setiap proses konfigurasi jaringan dan layanan server.

Hasil praktikum ini memiliki relevansi yang tinggi dengan kebutuhan administrasi server di lingkungan industri, mengingat SSH merupakan standar de facto untuk pengelolaan server berbasis Linux pada berbagai sektor, mulai dari penyedia layanan hosting, pusat data perusahaan, hingga infrastruktur komputasi awan. Kemampuan mengonfigurasi virtualisasi, container, jaringan, dan akses jarak jauh yang aman sebagaimana dipraktikkan pada laporan ini merupakan kompetensi dasar yang banyak dibutuhkan oleh tenaga administrator sistem dan DevOps di dunia kerja.

Dari sisi efisiensi pengelolaan server, implementasi SSH memberikan manfaat berupa kemudahan administrasi jarak jauh tanpa memerlukan kehadiran fisik administrator, kemampuan menjalankan banyak sesi manajemen secara bersamaan, serta dukungan otomatisasi tugas administratif melalui skrip maupun perangkat orkestrasi. Dengan demikian, implementasi SSH pada server Proxmox VE tidak hanya memenuhi aspek keamanan komunikasi, tetapi juga berkontribusi terhadap efisiensi waktu dan sumber daya dalam pengelolaan infrastruktur teknologi informasi secara keseluruhan.

Secara keseluruhan, praktikum ini berhasil mencapai tujuan yang telah ditetapkan, yaitu memahami proses konfigurasi server virtual, pembuatan Linux Container, konfigurasi jaringan, serta implementasi Secure Shell (SSH) menggunakan aplikasi PuTTY sebagai media akses jarak jauh ke server.

## **BAB V**

### **PENUTUP**

#### **5.1 Kesimpulan**

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa implementasi Secure Shell (SSH) pada server Proxmox VE menggunakan aplikasi PuTTY berhasil dilakukan dengan baik. Proses praktikum diawali dengan pembuatan mesin virtual pada Oracle VM VirtualBox, instalasi dan konfigurasi Proxmox VE, pembuatan Linux Container (LXC), serta konfigurasi jaringan untuk mendukung layanan SSH.

Layanan SSH berhasil diaktifkan pada Linux Container dan dapat diakses melalui aplikasi PuTTY menggunakan konfigurasi Port Forwarding yang telah ditentukan. Keberhasilan proses login menunjukkan bahwa koneksi SSH telah berjalan dengan baik sehingga pengguna dapat melakukan administrasi server secara aman melalui terminal. Dengan demikian, tujuan praktikum dalam menerapkan akses jarak jauh menggunakan protokol SSH pada lingkungan virtualisasi berbasis Proxmox VE telah tercapai.

Praktikum ini juga menunjukkan bahwa pemilihan Linux Container sebagai media implementasi memberikan efisiensi sumber daya yang signifikan dibandingkan Virtual Machine, tanpa mengurangi validitas hasil pengujian terhadap konsep dasar SSH, autentikasi, dan konfigurasi jaringan pada Proxmox VE. Konfigurasi Port Forwarding pada Oracle VM VirtualBox terbukti menjadi solusi efektif untuk menjembatani komunikasi antara komputer host dan layanan yang berjalan pada lingkungan virtual yang terisolasi.

Secara umum, seluruh rangkaian praktikum mulai dari instalasi platform virtualisasi, pembuatan lingkungan container, konfigurasi jaringan, hingga pengujian akses jarak jauh melalui SSH telah memberikan pemahaman yang komprehensif mengenai keterkaitan antara teknologi virtualisasi dan praktik keamanan administrasi server. Kompetensi yang diperoleh dari praktikum ini menjadi bekal yang relevan untuk diterapkan dalam pengelolaan infrastruktur teknologi informasi, baik pada skala akademik maupun dunia kerja di bidang administrasi sistem dan jaringan.

## 5.2 Saran

Pelaksanaan praktikum sebaiknya dilakukan dengan memperhatikan konfigurasi jaringan, terutama pengaturan alamat IP dan Port Forwarding, agar proses koneksi SSH dapat berjalan tanpa kendala. Berdasarkan hasil dan pembahasan yang telah diuraikan, terdapat beberapa rekomendasi teknis yang dapat diterapkan untuk meningkatkan keamanan dan efektivitas implementasi SSH pada lingkungan produksi, sebagai berikut.

- a. **Menerapkan autentikasi SSH Key** sebagai pengganti autentikasi berbasis kata sandi, karena pasangan kunci publik dan privat jauh lebih tahan terhadap serangan brute force maupun penembakan kata sandi.
- b. **Menonaktifkan login root melalui SSH** (`PermitRootLogin no`) agar akses administratif tertinggi tidak dapat langsung disasar oleh pihak yang tidak berwenang.
- c. **Mengganti port default SSH** dari port 22 ke port lain guna mengurangi eksposur terhadap pemindaian otomatis (automated scanning) yang umum menysasar port standar.
- d. **Menerapkan konfigurasi firewall** untuk membatasi alamat IP yang diperbolehkan mengakses layanan SSH sesuai kebutuhan administrasi.
- e. **Melakukan pembaruan sistem secara berkala** pada Proxmox VE maupun Linux Container guna menutup celah keamanan yang mungkin ditemukan pada versi perangkat lunak sebelumnya.
- f. **Melakukan monitoring log keamanan** secara rutin, khususnya log autentikasi SSH, guna mendeteksi dan merespons secara dini apabila terjadi percobaan akses yang mencurigakan.

Penerapan rekomendasi tersebut diharapkan dapat meningkatkan tingkat keamanan implementasi SSH pada server Proxmox VE, khususnya apabila konfigurasi serupa diterapkan pada lingkungan produksi yang menangani data maupun layanan yang lebih kritis dibandingkan lingkungan pengujian pada praktikum ini.

## DAFTAR PUSTAKA

- Barrett, D. J., Silverman, R. E., & Byrnes, R. G. (2005). *SSH: The Secure Shell: The Definitive Guide* (2nd ed.). O'Reilly Media.
- Bhattiprolu, S., Biederman, E. W., Hallyn, S., & Lezcano, D. (2008). *Virtual servers and checkpoint/restart in mainstream Linux*. ACM SIGOPS Operating Systems Review, 42(5), 104–113.
- Internet Engineering Task Force. (2006). *RFC 4251: The Secure Shell (SSH) protocol architecture*. <https://www.rfc-editor.org/rfc/rfc4251>
- Linux Containers Project. (2024). *Linux Containers*. <https://linuxcontainers.org/>
- Oracle Corporation. (2024). *Oracle VM VirtualBox User Manual*. <https://www.virtualbox.org/manual/>
- Proxmox Server Solutions GmbH. (2024). *Proxmox VE Administration Guide*. <https://pve.proxmox.com/pve-docs/>
- PuTTY Development Team. (2024). *PuTTY documentation*. <https://www.chiark.greenend.org.uk/~sgtatham/putty/docs.html>
- Rahman, A., & Kartika, D. (2022). *Analisis keamanan jaringan pada implementasi Secure Shell (SSH) untuk administrasi server jarak jauh*. Jurnal Teknik Informatika dan Sistem Informasi, 8(2), 215–224.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- VMware. (2023). *What Is a Virtual Machine?* <https://www.vmware.com/topics/virtual-machine>